comparative_analysis

January 27, 2026

[2]:
```
# ============================================================
# CLASSICAL vs DEEP LEARNING - UNIFIED COMPARATIVE ANALYSIS (PDF-SAFE)
# ============================================================
# PURPOSE:
# - Compare Classical ML and Deep Learning models fairly
# - Use identical core metrics
# - Provide performance-efficiency trade-off analysis
# - Display tables, plots, and detailed interpretations
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from IPython.display import display, Markdown

# ------------------------------------------------------------
# 1  PATH CONFIGURATION
# ------------------------------------------------------------
CODE_DIR = Path.cwd()
PROJECT_ROOT = CODE_DIR.parent

CLASSICAL_CSV = PROJECT_ROOT / "results/classical_comparison.csv"
DEEP_CSV = PROJECT_ROOT / "results/deep_learning_comparison_metrics.csv"

# ------------------------------------------------------------
# 2  CORE METRIC DEFINITION
# ------------------------------------------------------------
CORE_COLUMNS = [
    "Model",
    "Accuracy",
    "Precision (Macro)",
    "Recall (Macro)",
    "F1-score (Macro)",
    "Training Time (s)",
    "Inference Time / Sample (s)",
]
```

```
HIGHER_IS_BETTER = [
    "Accuracy",
    "Precision (Macro)",
    "Recall (Macro)",
    "F1-score (Macro)",
]

LOWER_IS_BETTER = [
    "Training Time (s)",
    "Inference Time / Sample (s)",
]

# ------------------------------------------------------------
# 3  LOAD CSV FILES
# ------------------------------------------------------------
classical_df = pd.read_csv(CLASSICAL_CSV)
deep_df = pd.read_csv(DEEP_CSV)

# ------------------------------------------------------------
# 4  VALIDATE CORE COLUMNS
# ------------------------------------------------------------
def validate_core_columns(df, name):
    missing = [c for c in CORE_COLUMNS if c not in df.columns]
    if missing:
        raise ValueError(f"  Missing columns in {name}: {missing}")
    core_order = [c for c in df.columns if c in CORE_COLUMNS]
    if core_order != CORE_COLUMNS:
        raise ValueError(
            f"  Core column order mismatch in {name}\n"
            f"Expected: {CORE_COLUMNS}\n"
            f"Found   : {core_order}"
        )
    print(f"  {name} validated")

validate_core_columns(classical_df, "Classical ML CSV")
validate_core_columns(deep_df, "Deep Learning CSV")

# ------------------------------------------------------------
# 5  UNIFIED BENCHMARK TABLE (PDF-SAFE)
# ------------------------------------------------------------
benchmark_df = pd.concat(
    [classical_df[CORE_COLUMNS], deep_df[CORE_COLUMNS]], ignore_index=True
)

display(Markdown("##   Unified Benchmark Metrics"))
display(benchmark_df.round(4))
```

```
# ------------------------------------------------------------
# 6  NORMALIZED SCORE INDEX (NSI)
# ------------------------------------------------------------
nsi_df = benchmark_df.copy()

# Normalize higher-is-better metrics
for col in HIGHER_IS_BETTER:
    min_v, max_v = nsi_df[col].min(), nsi_df[col].max()
    nsi_df[col + "_norm"] = (nsi_df[col] - min_v) / (max_v - min_v + 1e-9)

# Normalize lower-is-better metrics
for col in LOWER_IS_BETTER:
    min_v, max_v = nsi_df[col].min(), nsi_df[col].max()
    nsi_df[col + "_norm"] = (max_v - nsi_df[col]) / (max_v - min_v + 1e-9)

# Final NSI
norm_cols = [c for c in nsi_df.columns if c.endswith("_norm")]
nsi_df["NSI"] = nsi_df[norm_cols].mean(axis=1)
nsi_df = nsi_df.sort_values("NSI", ascending=False)

display(Markdown("##  Normalized Score Index (NSI)"))
display(nsi_df[["Model", "NSI"]].round(3))

# ------------------------------------------------------------
# 7  PLOT 1: ACCURACY vs TRAINING TIME (SCATTER PLOT)
# ------------------------------------------------------------
display(Markdown("##  Accuracy vs Training Time (Scatter Plot)"))

plt.figure(figsize=(8, 5))

benchmark_df["Model Type"] = ["Classical ML"] * len(classical_df) + ["Deep␣
 ↪Learning"] * len(deep_df)
marker_sizes = benchmark_df["Inference Time / Sample (s)"] * 2000
colors = {"Classical ML": "#1f77b4", "Deep Learning": "#ff7f0e"}

for mtype in benchmark_df["Model Type"].unique():
    subset = benchmark_df[benchmark_df["Model Type"] == mtype]
    plt.scatter(
        subset["Training Time (s)"],
        subset["Accuracy"],
        s=marker_sizes[subset.index],
        c=colors[mtype],
        label=mtype,
        alpha=0.7,
        edgecolors="k"
    )
```

```
for _, r in benchmark_df.iterrows():
    plt.text(r["Training Time (s)"] * 1.01, r["Accuracy"], r["Model"],␣
 ↪fontsize=9)

plt.xlabel("Training Time (s)")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Training Time\n(Color = Model Type, Marker size ␣
 ↪Inference Time)")
plt.legend(title="Model Type")
plt.grid(True)
plt.show()

# ------------------------------------------------------------
# 8  PLOT 2: INFERENCE TIME COMPARISON (HORIZONTAL BAR)
# ------------------------------------------------------------
display(Markdown("##  Inference Time per Sample (Latency Comparison)"))

sorted_df = benchmark_df.sort_values("Inference Time / Sample (s)",␣
 ↪ascending=True)

plt.figure(figsize=(8, 5))
plt.barh(sorted_df["Model"], sorted_df["Inference Time / Sample (s)"])

for i, (_, r) in enumerate(sorted_df.iterrows()):
    plt.text(r["Inference Time / Sample (s)"], i, f"Acc={r['Accuracy']:.3f}",␣
 ↪va="center", fontsize=9)

plt.xlabel("Inference Time / Sample (s)")
plt.title("Inference Latency Comparison\n(Annotated with Accuracy)")
plt.grid(axis="x")
plt.show()

# ------------------------------------------------------------
# 9  OVERALL NSI RANKING PLOT
# ------------------------------------------------------------
display(Markdown("##  Overall Model Ranking (NSI)"))

plt.figure(figsize=(8, 5))
plt.barh(nsi_df["Model"], nsi_df["NSI"])
plt.xlabel("Normalized Score Index (NSI)")
plt.gca().invert_yaxis()
plt.grid(axis="x")
plt.show()

# ------------------------------------------------------------
#  FINAL TEXTUAL ANALYSIS
```

```
# ------------------------------------------------------------
best = nsi_df.iloc[0]

display(Markdown(f"""
##  Comparative Analysis Summary

###  Best Overall Model
- **{best['Model']}**
- **NSI Score:** `{best['NSI']:.3f}`

###  Key Observations
- Deep learning models achieve high accuracy but incur significant training and␣
 ↪inference costs
- Classical ML models remain computationally efficient and are better suited␣
 ↪for real-time or edge deployment

###  Final Conclusion
Accuracy alone is insufficient for fair comparison.
The Normalized Score Index (NSI) provides a quantitative way to balance␣
 ↪performance and efficiency across different learning paradigms.
"""))

print("  Comparative analysis displayed successfully")
```

```
 Classical ML CSV validated
 Deep Learning CSV validated
```

0.1 Unified Benchmark Metrics

	Model	Accuracy	Precision (Macro)	Recall (Macro)	\
0	SVM	0.9167	0.9203	0.9167	
1	Random Forest	0.9333	0.9344	0.9333	
2	k-NN	0.9333	0.9331	0.9333	
3	2D CNN + Temporal Pooling	1.0000	1.0000	1.0000	
4	3D CNN (R(2+1)D)	1.0000	1.0000	1.0000	

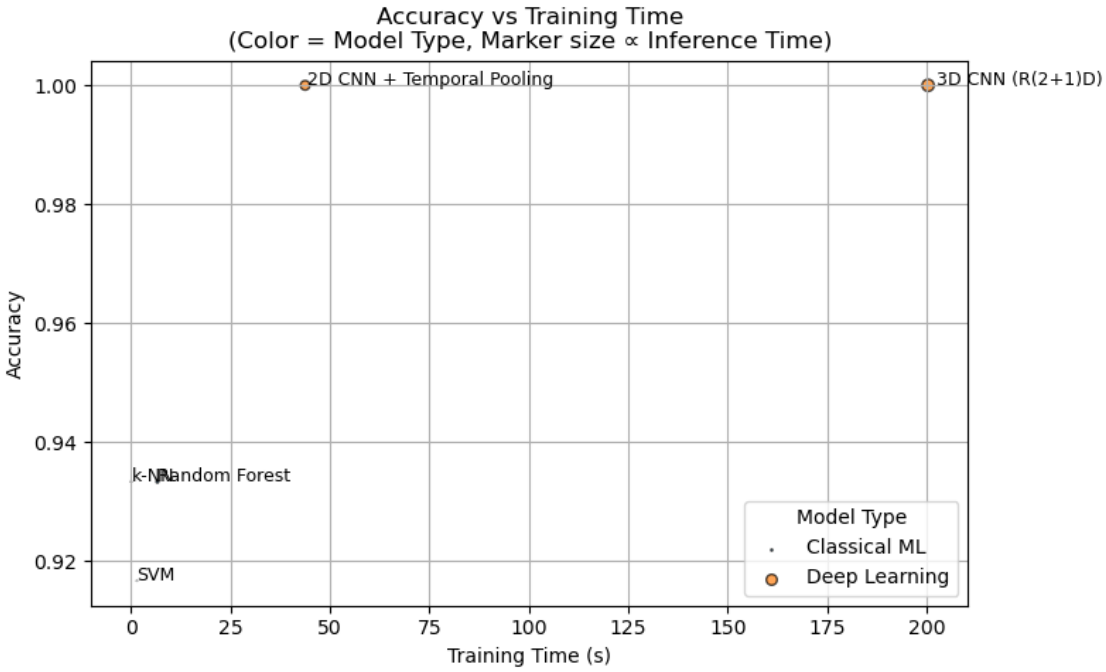
	F1-score (Macro)	Training Time (s)	Inference Time / Sample (s)
0	0.9157	1.5324	0.0000
1	0.9326	6.6875	0.0008
2	0.9326	0.1003	0.0001
3	1.0000	43.8125	0.0115
4	1.0000	200.3550	0.0188

0.2 Normalized Score Index (NSI)

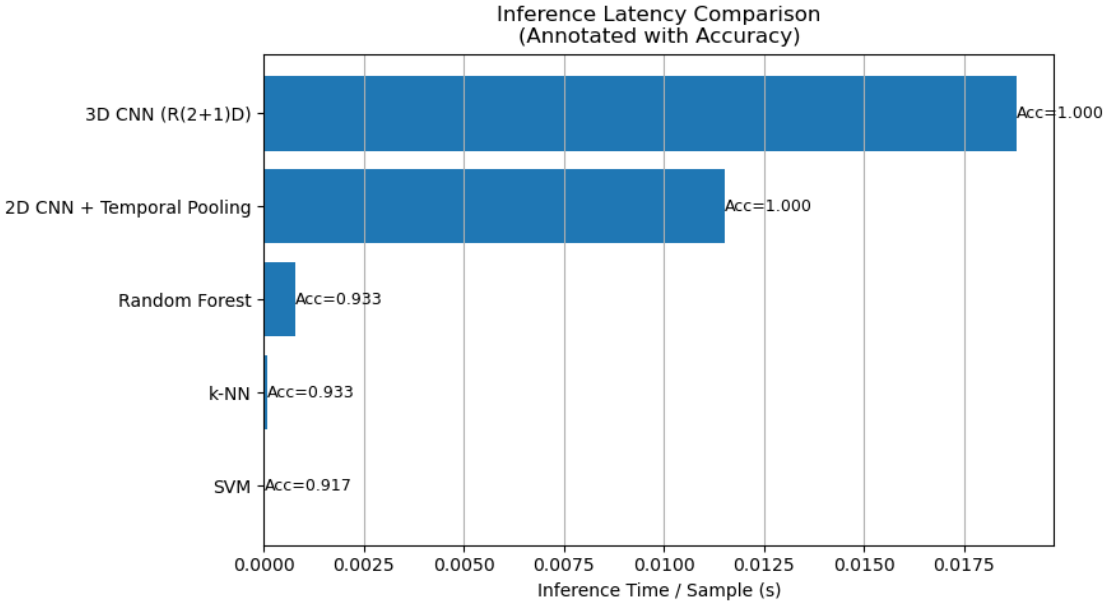
	Model	NSI
3	2D CNN + Temporal Pooling	0.862
4	3D CNN (R(2+1)D)	0.667

2	k-NN	0.460
1	Random Forest	0.450
0	SVM	0.332

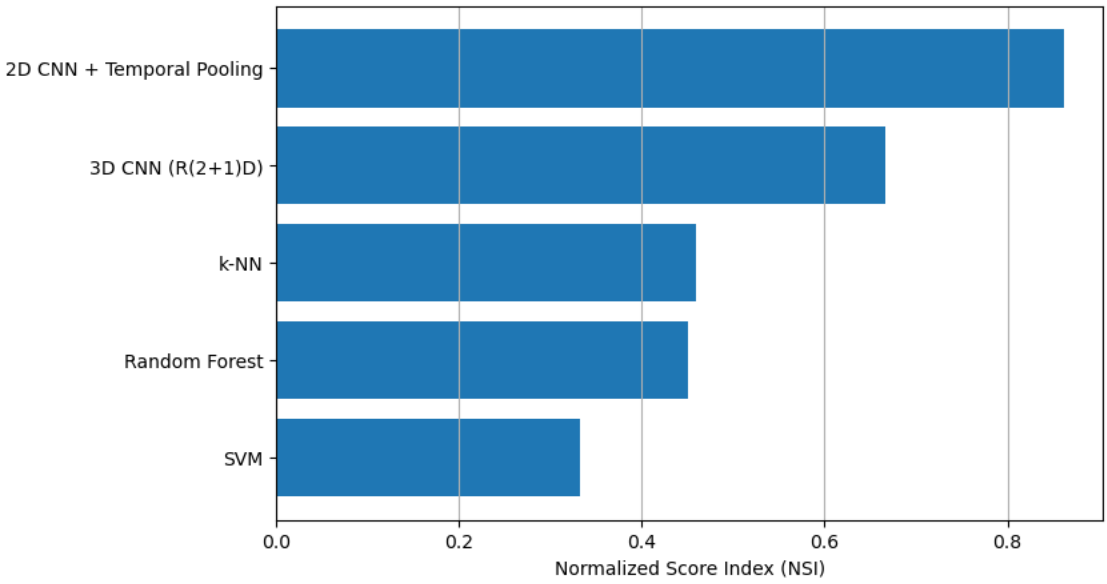
0.3 Accuracy vs Training Time (Scatter Plot)



0.4 Inference Time per Sample (Latency Comparison)



0.5 Overall Model Ranking (NSI)



0.6 Comparative Analysis Summary

0.6.1 Best Overall Model

- **2D CNN + Temporal Pooling**
- **NSI Score:** `0.862`

0.6.2 Key Observations

- Deep learning models achieve high accuracy but incur significant training and inference costs
- Classical ML models remain computationally efficient and are better suited for real-time or edge deployment

0.6.3 Final Conclusion

Accuracy alone is insufficient for fair comparison. The Normalized Score Index (NSI) provides a quantitative way to balance performance and efficiency across different learning paradigms.

```
Comparative analysis displayed successfully
```

[ ]: